WEEK 11: DATABASE AUTOMATION USING VIEWS AND PROCEDURES

To implement database automation in the Food ordering management Database System using Views, Stored Procedures, and Functions to simplify database operations and improve efficiency.

1. Customer Order Report View:

```
SQL> CREATE OR REPLACE VIEW CUSTOMER_ORDER_REPORT AS SELECT C.CUSTOMER_NAME, O.ORDER_ID,O.ORDER_DATE, O.TOTAL_AMOUNT
,O.ORDER_STATUS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID=C.CUSTOMER_ID;

View created.
```

2. Product Sales Report View:

```
SQL> CREATE OR REPLACE VIEW PRODUCT_SALES_REPORT AS
  2  SELECT P.PRODUCT_NAME,
  3  SUM(OD.QUANTITY) AS QUANTITY_SOLD,
  4  SUM(OD.QUANTITY * P.PRICE) AS TOTAL_REVENUE
  5  FROM PRODUCT P JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID
  6  GROUP BY P.PRODUCT_NAME;

View created.
```

3. Payment Status View:

```
SQL> CREATE OR REPLACE VIEW PAYMENT_STATUS_VIEW AS
  2  SELECT ORDER_ID, PAYMENT_MODE, PAYMENT_STATUS,TRANSACTION_AMOUNT
  3  FROM PAYMENT;

View created.
```

2. Develop Stored Procedure for Order Processing

1. Add New Customer Order:

- Insert new order information.
- Add purchased product details.
- Calculate the total order amount.

```
SQL> CREATE OR REPLACE PROCEDURE ADD_CUSTOMER_ORDER C
  2
  3  )
  4
  5  P_ORDER_ID IN VARCHAR2,
  6
  7  P_CUSTOMER_ID IN NUMBER,
  8
  9  P_ORDER_DETAIL_ID IN VARCHAR2,
 10
 11  P_PRODUCT_ID IN NUMBER,
 12
 13  P_QUANTITY IN NUMBER,
 14
 15  P_PAYMENT_ID IN NUMBER,
 16
 17  P_PAYMENT_MODE IN VARCHAR2
 18
 19  AS
 20
 21  V_PRICE NUMBER(10,2);
 22
 23  V_STOCK NUMBER(5);
 24
 25  V_TOTAL NUMBER(10,2);
 26
 27  BEGIN
 28
 29  SELECT PRICE, STOCK_QUANTITY INTO V_PRICE, V_STOCK
 30
 31  FROM PRODUCT
 32
 33  WHERE PRODUCT_ID = P_PRODUCT_ID;
 34  IF P_QUANTITY <= 0 THEN
 35
 36  RAISE_APPLICATION_ERROR(-20001, 'Invalid quantity'); END IF;
 37
 38  IF V_STOCK < P_QUANTITY THEN
 39
 40  RAISE_APPLICATION_ERROR(-20002, 'Insufficient stock'); END IF;
 41
 42  V_TOTAL : V_PRICE P_QUANTITY;
 43
 44  INSERT INTO ORDERS (ORDER_ID, CUSTOMER_ID, ORDER_DATE, TOTAL_AMOUNT, ORDER_STATUS)
 45
 46  VALUES (P_ORDER_ID, P_CUSTOMER_ID, SYSDATE, V_TOTAL, 'Pending');
 47
 48  INSERT INTO ORDER_DETAILS (ORDER_DETAIL_ID, ORDER_ID, PRODUCT_ID, QUANTITY, PRICE)
 49
 50  VALUES
 51  (P_ORDER_DETAIL_ID, P_ORDER_ID, TO_CHAR(P_PRODUCT_ID), P_QUANTITY, V_PRICE);
 52
 53  UPDATE PRODUCT
 54
 55  SET STOCK_QUANTITY = STOCK_QUANTITY P_QUANTITY
 56
 57  WHERE PRODUCT_ID = P_PRODUCT_ID;
 58
 59  INSERT INTO PAYMENT
 60
 61  (PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT MODE, PAYMENT_STATUS, TRANSACTION_AMOUNT)
 62
 63  VALUES
 64
 65  (P_PAYMENT_ID, P_ORDER_ID, SYSDATE, P_PAYMENT_MODE, 'Pending', V_TOTAL);
 66
 67  DBMS_OUTPUT.PUT_LINE('Order Added Successfully'); DBMS_OUTPUT.PUT_LINE('Total Amount: ' || V_TOTAL);
 68
 69  EXCEPTION
 70
 71  WHEN NO_DATA_FOUND THEN
 72
 73  RAISE_APPLICATION_ERROR(-20003, 'Product not found');
 74
 75  END;
 76
 77  /

Warning: Procedure created with compilation errors.
```

2. Update Order Status

Example:

Change: Pending → Shipped → Delivered

```
SQL> CREATE OR REPLACE PROCEDURE UPDATE_ORDER_STATUS
  2  (
  3  P_ORDER_ID IN VARCHAR2,
  4  P_STATUS IN VARCHAR2
  5  )
  6  AS
  7  BEGIN
  8  UPDATE ORDERS
  9  SET ORDER_STATUS=P_STATUS
 10  WHERE ORDER_ID=P_ORDER_ID;
 11  IF SQL%ROWCOUNT=0 THEN
 12  DBMS_OUTPUT.PUT_LINE('ORDER NOT FOUND');
 13  ELSE
 14  DBMS_OUTPUT.PUT_LINE('ORDER STATUS UPDATED SUCCESSFULLY');
 15  END IF;
 16  END;
 17  /

Procedure created.
```

```
SQL> EXEC UPDATE_ORDER_STATUS('O002','DELIVERED');

PL/SQL procedure successfully completed.

SQL> SELECT * FROM ORDERS;

ORDER_I CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
------- ----------- --------- ------------ --------------
O001              1 01-SEP-26         2500 DELIVERED
O002              2 02-SEP-26         1800 DELIVERED
O003              1 03-SEP-26         3200 DELIVERED
O004              3 04-SEP-26         1500 PENDING
O005              4 05-SEP-26         4200 DELIVERED
O006              2 06-SEP-26         2750 SHIPPED
O007              5 07-SEP-26         3600 DELIVERED
O008              6 08-SEP-26         1200 PENDING
O009              1 09-SEP-26         5100 DELIVERED
O010              7 10-SEP-26         2300 SHIPPED
O011              8 11-SEP-26         1900 DELIVERED

ORDER_I CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
------- ----------- --------- ------------ --------------
O012              3 12-SEP-26         3400 PENDING
O013              9 13-SEP-26         2800 SHIPPED
O014             10 14-SEP-26         4500 DELIVERED
O015              4 15-SEP-26         1600 PENDING

15 rows selected.
```

3. Generate Customer Purchase History

Input: Customer_ID Output:

- Order details
- Purchase amount
- Order status

```
SQL> CREATE OR REPLACE PROCEDURE CUSTOMER_PURCHASE_HISTORY
  2
  3  P_CUSTOMER_ID IN NUMBER
  4
  5  )
  6
  7  AS BEGIN
  8
  9  FOR R IN
 10
 11  C
 12
 13  SELECT ORDER_ID ORDER_DATE, TOTAL AMOUNT, ORDER STATUS FROM ORDERS WHERE CUSTOMER_ID = P_CUSTOMER_ID
 14
 15  )
 16
 17  LOOP
 18
 19  DBMS_OUTPUT.PUT_LINE( 'Order ID: || R. ORDER_ID );
 20
 21  DBMS_OUTPUT.PUT_LINE( 'Order Date: || R.ORDER_DATE );
 22
 23  DBMS_OUTPUT.PUT_LINE( ); 'Purchase Amount: || R. TOTAL_AMOUNT
 24
 25  DBMS_OUTPUT.PUT_LINE( 'Order Status: || R.ORDER_STATUS );
 26
 27  END LOOP;
 28
 29  END;
 30  /

Warning: Procedure created with compilation errors.
```

3. Create Functions for Sales Calculations

1. Calculate Total Sales Input:

- Date range Output:
- Total revenue generated

```
SQL> CREATE OR REPLACE FUNCTION CALCULATE_TOTAL_SALES
  2
  3  P_START_DATE IN DATE, P_END_DATE IN DATE
  4
  5  )
  6
  7  RETURN NUMBER
  8
  9  AS
 10
 11  V_TOTAL NUMBER;
 12
 13  BEGIN
 14
 15  2
 16
 17  4
 18
 19  S
 20
 21  5
 22
 23  SELECT NVL(SUM(TOTAL_AMOUNT), 0) INTO V_TOTAL FROM ORDERS WHERE OR
DER_DATE >= TRUNC(P_START_DATE) AND ORDER_DATE < TRUNC(P_END_DATE) + 1;

 24
 25  RETURN V_TOTAL;
 26
 27  END;
 28  /

Warning: Function created with compilation errors.
```

2. Calculate Customer Total Spending

Input:

● Customer_ID Output:

● Total amount spent

```
SQL> CREATE OR REPLACE FUNCTION PRODUCT_REVENUE
  2  (
  3  P_PRODUCT_ID IN NUMBER
  4  )
  5  RETURN
  6  AS
  7  V_REVENUE NUMBER;
  8  BEGIN
  9  SELECT NVL(SUM(QUANTITY*PRICE),0)
 10  INTO V_REVENUE
 11  FROM ORDER_DETAILS
 12  WHERE PRODUCT_ID=TO_CHAR(P_PRODUCT_ID);
 13  RETURN V_REVENUE;
 14  END;
 15  /

Warning: Function created with compilation errors.
```

4. Automate Repetitive Database Operations

Order Processing Automation

When a customer places an order:

1. Insert order details.
2. Reduce product stock.
3. Create payment record.
4. Update order status.

```
SQL> CREATE OR REPLACE TRIGGER TRG_REDUCE_STOCK
  2  AFTER INSERT ON ORDERS
  3  FOR EACH ROW
  4  BEGIN
  5  UPDATE PRODUCT SET STOCK_QUANTITY = STOCK_QUANTITY - :NEW.QUANTITY

  6  WHERE PRODUCT_ID = :NEW.PRODUCT_ID;
  7  END;
  8  /

Warning: Trigger created with compilation errors.
```

```
SQL> CREATE OR REPLACE PROCEDURE RESTOCK_PRODUCT (p_prod_id NUMBER, p_q
ty NUMBER) IS
  2  BEGIN
  3    UPDATE PRODUCT SET STOCK_QUANTITY = STOCK_QUANTITY + p_qty WHERE
 PRODUCT_ID = p_prod_id;
  4    COMMIT;
  5  END;
  6  /

Procedure created.
```

2. Inventory Update Automation:

The inventory operations include:

• Increase stock after receiving new supply.

• Reduce stock after a purchase.

• Identify products with low stock.

```
SQL> CREATE OR REPLACE VIEW INVENTORY_REPORT AS
  2  SELECT PRODUCT_NAME, STOCK_QUANTITY AS STOCK,
  3         CASE WHEN STOCK_QUANTITY < 10 THEN 'Low Stock' ELSE 'In Sto
ck' END AS STATUS
  4  FROM PRODUCT;

View created.
```

3. Report Generation Automation:

The system generates the following reports:

• Daily sales report.

```
SQL> CREATE OR REPLACE VIEW DAILY_SALES_REPORT AS
  2  SELECT TRUNC(ORDER_DATE) AS SALE_DATE, COUNT(*) AS ORDERS, SUM(TOT
AL_AMOUNT) AS REVENUE
  3  FROM ORDERS GROUP BY TRUNC(ORDER_DATE);

View created.
```

• Monthly revenue report.

```
SQL> CREATE OR REPLACE VIEW MONTHLY_REVENUE_REPORT AS
  2  SELECT TO_CHAR(ORDER_DATE,'YYYY-MM') AS MONTH, SUM(TOTAL_AMOUNT) A
S REVENUE
  3  FROM ORDERS GROUP BY TO_CHAR(ORDER_DATE,'YYYY-MM');

View created.
```

• Customer purchase report.

```
SQL> CREATE OR REPLACE VIEW CUSTOMER_PURCHASE_REPORT AS
  2  SELECT C.CUSTOMER_NAME AS CUSTOMER, COUNT(O.ORDER_ID) AS ORDERS, S
UM(O.TOTAL_AMOUNT) AS AMOUNT
  3  FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID
  4  GROUP BY C.CUSTOMER_NAME;

View created.
```